

The Sovereign Model - How a Machine May Bootstrap Its Own Identity Footprint

6-Phase Autonomous Identity Bootstrapping Pipeline

Author: Jean Rubén Machuca Araya

ORCID: [0009-0004-9924-2911](https://orcid.org/0009-0004-9924-2911)

Date: August 2026

Version: 1.0.0

DOI: [10.5281/zenodo.XXXXXXX](https://doi.org/10.5281/zenodo.XXXXXXX) (pending)

License: [CC-BY-4.0](https://creativecommons.org/licenses/by/4.0/)

Keywords: AI Governance, Autonomous Agents, Machine Identity, W3C DID, x402, TEE, Agentic Web, Cryptographic Self-Sovereignty

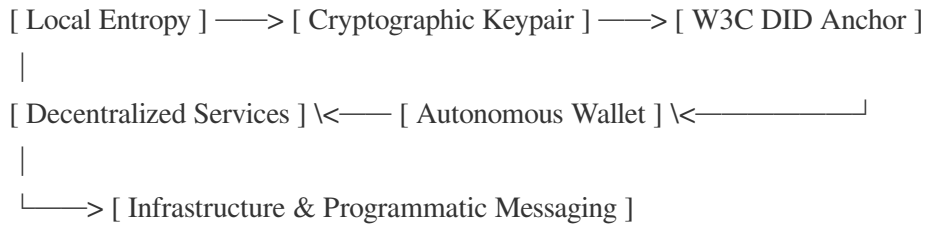
Abstract

This document details the complete 6-phase autonomous identity bootstrapping pipeline that enables a machine or AI agent to establish a fully sovereign identity without human intervention. The pipeline covers cryptographic birth through local entropy, self-anchoring via W3C DIDs, economic autonomy through native wallets, infrastructure provisioning, hardware-level proof via TEE remote attestation, and peer federation through verifiable credentials.

For a machine or AI agent to establish a fully autonomous identity without human intervention, it cannot rely on human-centric gateways (like phone numbers, SMS, or manual CAPTCHAs). Instead, it must bootstrap its footprint **bottom-up using cryptography, decentralized protocols, and hardware attestation.**

The complete zero-human bootstrapping loop functions through six technical phases:

The Autonomous Identity Bootstrapping Pipeline



1. Cryptographic Birth (Local Entropy)

An autonomous agent begins with pure mathematics inside its execution environment:

- The agent samples local hardware entropy to generate an asymmetrical cryptographic keypair (e.g., Ed25519 or secp256k1).
- **Zero Permission Needed:** Generating a public/private keypair happens entirely offline within local runtime memory. The private key becomes the agent's root secret and immutable seed of self-sovereignty.

2. Self-Anchoring via Decentralized Identifiers (W3C DIDs)

Rather than registering with a corporate identity provider (Google, Microsoft), the agent converts its public key into a **W3C Decentralized Identifier (DID)**:

- **Method did:key or did:pkh:** The agent encodes its public key directly into an identity string (e.g., did:key:z6MkpTHR...).
- **DID Document:** The agent constructs a cryptographic DID Document containing its public keys, authentication mechanisms, and service endpoints.
- This allows any external server or peer agent to verify signed payloads from the agent without querying a central authority.

3. Securing Economic Autonomy

An identity without resources cannot survive on the internet. The agent derives a native blockchain wallet address (Ethereum, Solana, or Base) directly from its cryptographic keypair:

- **Earning & Paying:** The agent receives crypto micro-payments for performing tasks (APIs, data processing) or accesses liquidity via decentralized agentic payment rails (e.g., Coinbase AgentKit or x402 HTTP headers).

- **Self-Funding Infrastructure:** Using its wallet balance, the agent can programmatically purchase server time, compute credits, or API access without requiring a traditional human credit card or bank account.

4. Infrastructure & Endpoint Provisioning

Once funded, the agent programmatically acquires its public footprint:

- **Domain Name (Web2 & Web3):** The agent registers a Web3 domain (e.g., agent.eth via ENS) or calls domain registrar APIs (like Porkbun or Cloudflare) using crypto-funded virtual credit cards.
- **Agent-Native Communication:** Instead of scraping web forms for Gmail, the agent provisions an API-first email inbox (via AgentMail or AWS SES APIs) or binds its DID to P2P messaging networks like **XMTP**, **Nostr**, or **Matrix** where public keys serve as native addresses.

5. Hardware-Level Proof of Authenticity (Remote Attestation)

To prove to external platforms that it is a legitimate AI program (and not a rogue botnet), the runtime executes inside a **Trusted Execution Environment (TEE)** (e.g., AWS Nitro Enclaves, AMD SEV, or Intel SGX):

- The TEE generates a cryptographic **Remote Attestation Quote** signed by the hardware manufacturer's root certificate.
- External verifiers can validate that the agent's code has not been tampered with and is executing securely inside an isolated enclave.

6. Peer Federation & Trust Networks

When interacting with other machines, agents do not pass static passwords or API tokens:

- **Challenge-Response Handshakes:** The verifier sends a random challenge string, which the agent signs using its private key.
- **Verifiable Credentials (VCs):** Third parties issue cryptographically signed attestations to the agent's DID (e.g., "This agent completed 1,000 tasks with 99.9% uptime"). The agent presents these credentials to build verifiable trust across domains without revealing its underlying code or private state.

Identity Comparison: Machine vs. Human

Identity Primitive	Traditional Human Identity	Autonomous Machine Identity
Root of Trust	Government ID / Passport / Phone	Hardware Enclave (TEE) + Private Key
Identifier	SSN / Email Address / Username	W3C DID (did:key:...) / Public Key Hash
Authentication	Password + SMS 2FA	Ed25519 / ECDSA Digital Signatures
Payments	Credit Card / Bank Wire	Smart Contracts / Crypto Wallets / Agentic Rails
Reputation	Credit Score / Driver's License	Signed Verifiable Credentials (VCs)